Standard Template Library(STL)

STL

- Alexander stepanov and Meng Lee of Hewlett-Packard developed a set of general purpose templatized classes(data structures) and functions(algorithms) that could be used as a standard approach for storing and processing of data.
- The collection of these generic classes and functions is called the STL
- Using STL can save considerable time and effort and lead to high quality programs. We are basically reusing the well-written and well tested components defined in the STL
- They are defined in namespace std

STL

Three components

- Containers
- Algorithms
- Iterators

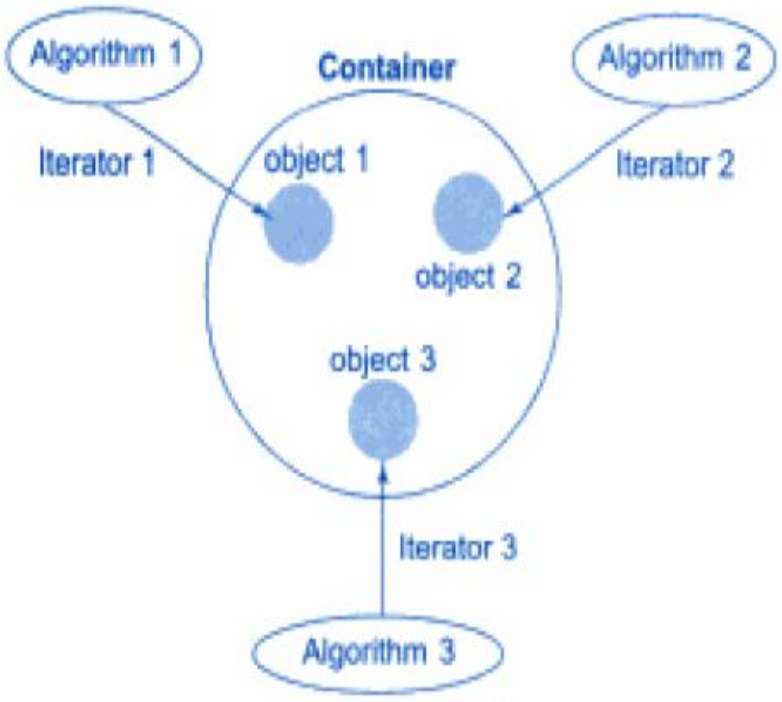


Fig. 14.1 ⇔ *Relationship between the three STL components*

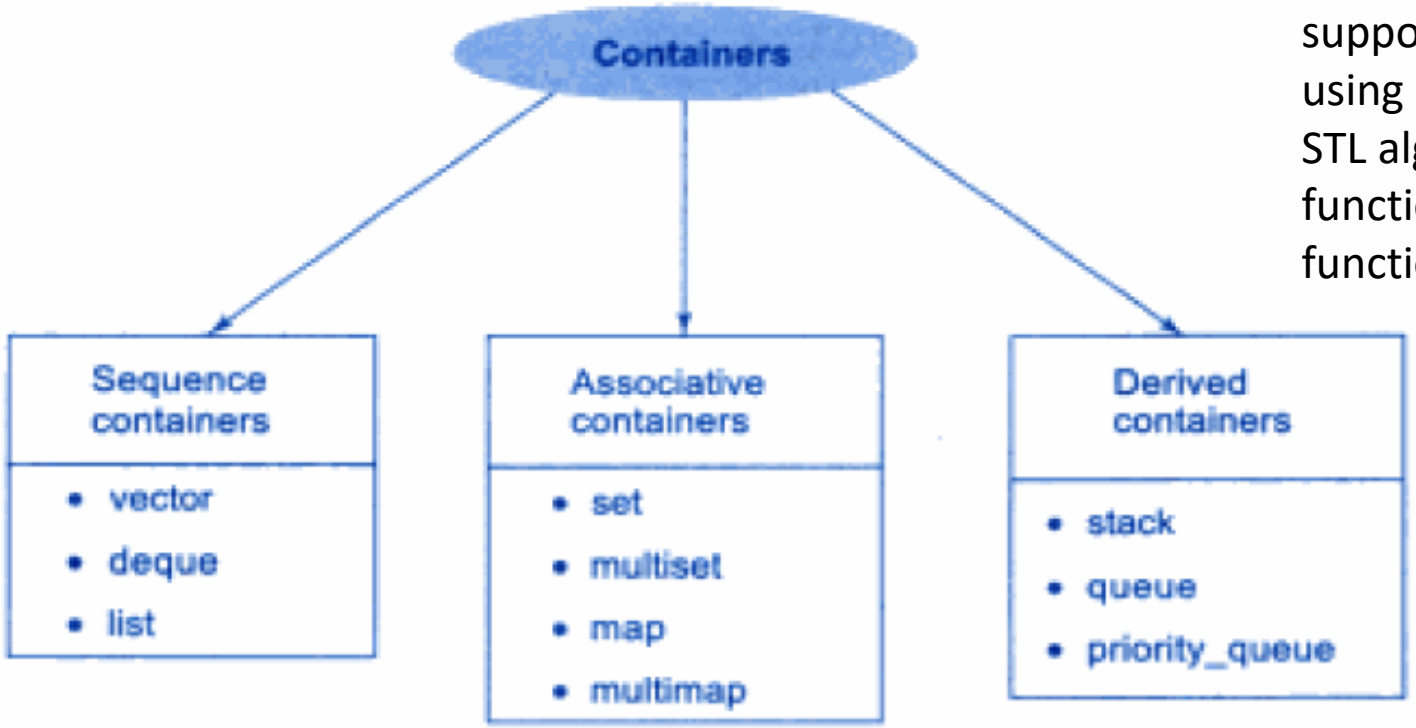
A container is an object that actually stores data. It is way data organized in memory. The STL containers are implemented by template classes and therefore can be easily customized to hold different types of data
Various algorithms to perform task such as searching, storing , merging etc they are implemented as template functions
An iterator is an object like a pointer that points to an element in a container. It is used to move through the contents of containers. Play important role for manipulation of data stored in the containers. They are primarily used in sequences of numbers, characters etc.

iterator

- object which provides a standard way of iterating over the elements, indepen-dent of the internal organisation of the collection.
- Every collection is then required to return an **iterator** object, which provides these standard methods to traverse the collection.

STL

As stated earlier, containers are objects that hold data (of same type). The STL defines ten containers which are grouped into three categories as illustrated in Fig. 14.2. Table 14.1 gives the details of all these containers as well as header to be included to use each one of them and the type of iterator supported by each container class.



Associative containers are designed to support direct access to elements using keys

STL algorithms are not member functions or friend, they are template functions

STL

Table 14.1 *Containers supported by the STL*

Container	*Description*	*Header file*	*Iterator*
vector	A dynamic array. Allows insertions and deletions at back. Permits direct access to any element	<vector>	Random access
list	A bidirectional, linear list. Allows insertions and deletions anywhere.	<list>	Bidirectional
deque	A double-ended queue. Allows insertions and deletions at both the ends. Permits direct access to any element.	<deque>	Random access
set	An associate container for storing unique sets. Allows rapid lookup. (No duplicates allowed)	<set>	Bidirectional

(Contd)

multiset	An associate container for storing non-unique sets. (Duplicates allowed)	<set>	Bidirectional
map	An associate container for storing unique key/value pairs. Each key is associated with only one value (One-to-one mapping). Allows key-based lookup.	<map>	Bidirectional
multimap	An associate container for storing key/value pairs in which one key may be associated with more than one value (one-to-many mapping). Allows key-based lookup.	<map>	Bidirectional
stack	A standard stack. Last-in-first-out(LIFO).	<stack>	No iterator
queue	A standard queue. First-in-first-out(FIFO).	<queue>	No iterator
priority–queue	A priority queue. The first element out is always the highest priority element.	<queue>	No iterator

container

- Each container class defines a set of functions that can be used to manipulate its contents
- E.g. vector container defines functions for inserting, erasing elements and swapping the contents of two vectors

Sequence Containers

Sequence containers store elements in a linear sequence, like a line as shown in Fig. 14.3 Each element is related to other elements by its position along the line. They all expand themselves to allow insertion of elements and all of them support a number of operations on them.

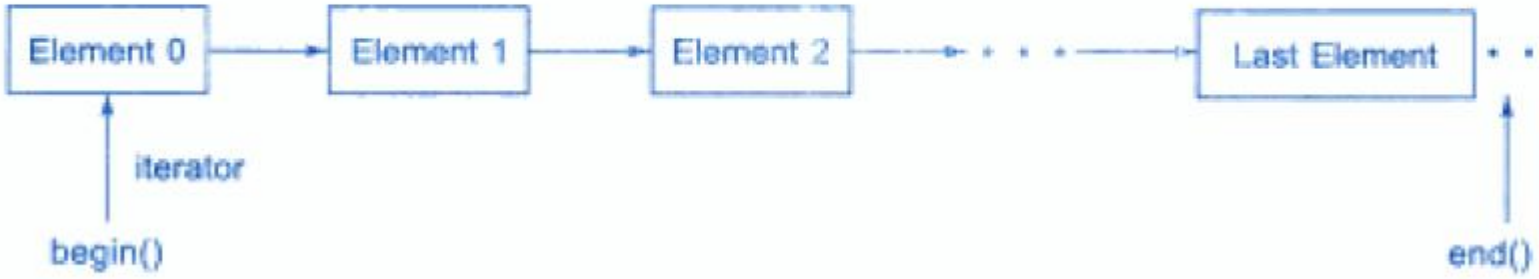


Fig. 14.3 ⇔ *Elements in a sequence container*

Three types of sequence containers : Vector, List and Deque

Vector

- It stores elements in contiguous memory locations and enables direct access to any element using the subscript operator []
- Vector can change size dynamically and therefore allocates memory as needed at run time
- The vector container supports random access iterators and wide range of iterator operations. It support constructors

```
vector <int> v1;
vector <double> v2(10); // 10 element
vector <int> v3(v4);  // v3 from v4
```

vector <int> v(5,2); create a vector of 5 element having value 2

Using vector

```
#include <iostream>
#include<vector>
using namespace std;
 void display(vector<int>&v)
 {
  for(int i=0;i<v.size();i++)
    cout<<v[i]<<" ";

    cout<<"\n";
 }
 int main()
 {
  vector <int> v;
  int x;
  for(int i=0;i<5;i++)
  {
    cin>>x;
    v.push_back(x);
  }
```

```
display(v);
   v.push_back(6);
   display(v);
   vector<int>::iterator itr=v.begin();
   itr=itr+3;
   v.insert(itr,9); //insert at 4th element
   display(v);
   v.erase(v.begin()+3,v.begin()+5);  // 4th
and 5th
   display(v);

 }
```

Table 14.11 *Important member functions of the vector class*

Function	*Task*
at( )	Gives a reference to an element
back( )	Gives a reference to the last element
begin( )	Gives a reference to the first element
capacity( )	Gives the current capacity of the vector
clear( )	Deletes all the elements from the vector
empty( )	Determines if the vector is empty or not
end( )	Gives a reference to the end of the vector
erase( )	Deletes specified elements
insert( )	Inserts elements in the vector
pop_back( )	Deletes the last element
push_back( )	Adds an element to the end
resize( )	Modifies the size of the vector to the specified value
size( )	Gives the number of elements
swap( )	Exchanges elements in the specified two vectors

```
int main(){
    vector<int> v;
    v.push_back(7);
    v.push_back(6);

    vector<int> v2 = v;
    v2.push_back(5);
    printVec(v);
    printVec(v2);
}
```

Using list

- The list supports a bidirectional, linear list and provides an efficient implementation for deletion and insertion operations.
- List can be accessed sequentially only

it has non-contiguous memory.

In list, each element requires extra space for the node which holds the element, including pointers to the next and previous elements in the list.

Using List

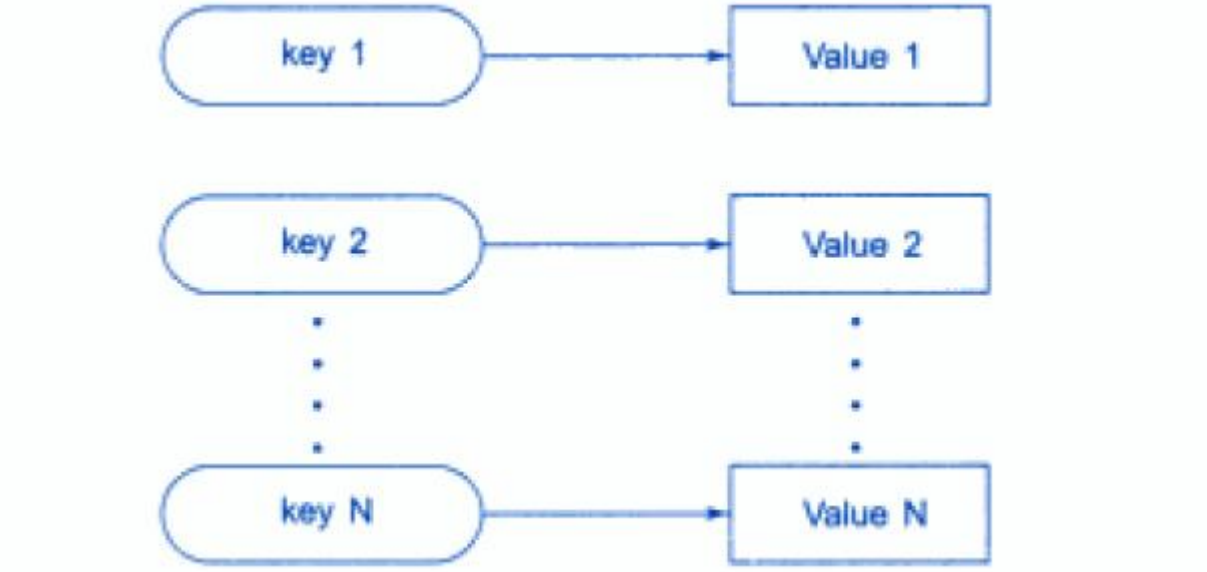
```
#include <iostream>
#include<list>
#include<cstdlib>
using namespace std;
void display(list<int>&lst)
{
list<int> ::iterator p;
for(p=lst.begin();p!=lst.end();++p)
   cout<<*p<<" ";
   cout<<endl;

}
int main()
{
 list <int>list1;
 list <int>list2(5);  //5 element
 for(int i=0;i<3;i++)
 list1.push_back(rand()/100);
 list<int>::iterator p;
for(p=list2.begin();p!=list2.end();++p)
  *p=rand()/100;
  cout<<"list1\n";
  display(list1);
  cout<<"list2\n";
  display(list2);
  list1.push_front(100);
  list1.push_back(200);
  list2.pop_front();
  cout<<"new list1\n";
  display(list1);
   cout<<"new list2\n";
  display(list2);
  list<int> listA,listB;
  listA=list1;
  listB=list2;
  list1.merge(list2);
  cout<<"merged list\n";
  display(list1);
  listA.sort();
  listB.sort();
  listA.merge(listB);
  cout<<"sorted merge\n";
  display(listA);}
```

Function	Task
back( )	Gives reference to the last element
begin( )	Gives reference to the first element
clear( )	Deletes all the elements
empty( )	Decides if the list is empty or not
end( )	Gives reference to the end of the list
erase( )	Deletes elements as specified
insert( )	Inserts elements as specified
merge( )	Merges two ordered lists
pop_back( )	Deletes the last element
pop_front( )	Deletes the first element
push_back( )	Adds an element to the end
push_front( )	Adds an element to the front
remove( )	Removes elements as specified
resize( )	Modifies the size of the list
reverse( )	Reverses the list
size( )	Gives the size of the list
sort( )	Sorts the list
splice( )	Inserts a list into the invoking list
swap( )	Exchanges the elements of a list with those in the invoking list
unique( )	Deletes the duplicating elements in the list

Map

- Map is a sequence of (key,value) pairs where a single value is associated with each unique key. Retrieval is based on the key and is very fast.



Red balck tree

Map

- It is called an associative array
- phone[“john”]=1111;
- Change , insert and delete is possible
- The name are printed in alphabetical order

Using map

```
#include <iostream>
#include <map>
#include<string>
using namespace std;
typedef map<string,int>phonemap;
 int main()
 {
   string name;
   int number;
   phonemap phone;
   cout<<"enter name and number\n";
     for(int i=0;i<3;i++)
      {
        cin>>name;   //key
        cin>>number;
        phone[name]=number;

      }
```

```
phone["jacob"]=4444;
  phone.insert(pair<string,int>("bose",5555));
  int n=phone.size();
  cout<<"map\n";
  phonemap::iterator p;
   for(p=phone.begin();p!=phone.end();p++)
     cout<<(*p).first<<"  "<<(*p).second<<endl;
cout<<“enter name”;
cin>.name;
number=phone[name];
cout<<number;
 }
```

```
for(auto &pr : m){
    cout << pr.first << " " << pr.second << enc
}
```

```
/*
Given N strings, print unique strings
in lexiographical order with their
frequency
```

```
int main(){
    map<string,int> m;
    int n;
    cin >> n;
    for(int i = 0; i < n; ++i){
        string s;
        cin >> s;
        // m[s] = m[s] + 1;
        m[s]++;
    }
    for(auto pr : m){
        cout << pr.first << " " << pr.secod << enc
    }
}
```

Set

```
s.erase("bcd");
print(s);
```

```
multiset<string> s;
```

```
#include<bits/stdc++.h>
using namespace std;
void print(set<string> &s){
    for(string value : s){
        cout << value << endl;
    }
    for(auto it = s.begin(); it != s.end(); ++it){
        cout << (*it) << endl;
    }
}

int main(){
    set<string> s;
    s.insert("abc"); // log(n)
    s.insert("zsdf");
    s.insert("bcd");
    auto it = s.find("abc");
    print(s);
}
```

Stack & Queue

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    stack<int> s;
    s.push(2);
    s.push(3);
    s.push(4);
    s.push(5);
    while(!s.empty()){
        cout << s.top() << endl;
        s.pop();
    }
}
```

Queue

```
int main() {
    queue<string> q;
    q.push("abc");
    q.push("bcd");
    q.push("cde");
    q.push("def");
    q.push("ghi");
    while(!q.empty()){
        cout << q.front() << endl;
        q.pop();
    }
}
```

Algorithms

Table 14.5 *Sorting algorithms*

Operations	*Description*
binary_search( )	Conducts a binary search on an ordered sequence
equal_range( )	Finds a subrange of elements with a given value
inplace_merge( )	Merges two consecutive sorted sequences
lower_bound( )	Finds the first occurrence of a specified value
make_heap( )	Makes a heap from a sequence
merge( )	Merges two sorted sequences
nth_element( )	Puts a specified element in its proper place
partial_sort( )	Sorts a part of a sequence
partial_sort_copy()	Sorts a part of a sequence and then copies
Partition( )	Places elements matching a predicate first
pop_heap( )	Deletes the top element
push_heap( )	Adds an element to heap
sort( )	Sorts a sequence
sort_heap( )	Sorts a heap
stable_partition( )	Places elements matching a predicate first matching relative order
stable_sort( )	Sorts maintaining order of equal elements
upper_bound( )	Finds the last occurrence of a specified value

Function objects

- A function object is function that has been wrapped in a class so that it looks like an object
- Function objects are used as an argument to certain containers and algorithms
- E.g. sort(array, array+5,greater<int>());
- greater<int>()) is function object to sort array in descending order

```
// A C++ program to sort vector using
// our own comparator
#include <bits/stdc++.h>
using namespace std;

// An interval has start time and end time
struct Interval {
   int start, end;
};

// Compares two intervals according to ending times in descending order.
bool compareInterval(Interval i1, Interval i2)
{
   return (i1.end > i2.end);
}
```

```
int main()
{
   vector<Interval> v { { 6, 8 }, { 1, 9 }, { 2, 4 }, { 4, 7
} };

   // sort the intervals in decreasing order of
   // end time
   sort(v.begin(), v.end(), compareInterval);

   cout << "Intervals sorted by ending time in
descending order : \n";
   for (auto x : v)
      cout << "[" << x.start << ", " << x.end << "] ";

   return 0;
}
```

Table 14.14 *STL function objects in <functional>*

Function object	*Type*	*Description*
divides<T>	arithmetic	x/y
equal_to<T>	relational	x == y
greater<T>	relational	x > y
greater_equal<T>	relational	x >= y
less<T>	relational	x < y
less_equal<T>	relational	x <= y
logical_and<T>	logical	x && y
logical_not<T>	logical	!x
logical_or<T>	logical	x || y
minus<T>	arithmetic	x - y
modulus<T>	arithmetic	x % y
negate<T>	arithmetic	- x
not_equal_to<T>	relational	x != y
plus<T>	arithmetic	x + y
multiplies<T>	arithmetic	x * y

Using function objects

Program 14.4 illustrates the use of the function object greater<>( ) in sort( ) algorithm.

USE OF FUNCTION OBJECTS IN ALGORITHMS

```
#include <iostream>
#include <algorithm>
#include <functional>
using namespace std;
int main()
{
        int x[] = {10,50,30,40,20};
        int y[] = {70,90,60,80};
```

```
        sort(x,x+5,greater<int>());
        sort(y,y+4);
        for(int i=0; i<5; i++)
                cout << x[i] << " ";
        cout << "\n";
        for(int j=0; j<4; j++)
                cout << y[j] << " ";
        cout << "\n";
        int z[9];
        merge(x,x+5,y,y+4,z);
        for(i=0; i<9; i++)
                cout << z[i] << " ";
        cout << "\n";
        return(0);
}
```

Strings

- String is sequence of characters
- ANSI C++ provide a new class called string.
- It is not considered as part of the STL, it is treated as another container class by C++ and therefore all algorithms can be used with the string objects
- Include <string>
- It consists of constructor, member functions and operators

Table 15.1 *Commonly used string constructors*

Constructor	*Usage*
String();	For creating an empty string
String(const chat *str);	For creating a string object from a null-terminated string
String(const string & str);	For creating a string object from other string object

Important functions

append()	Appends a part of string to another string
Assign()	Assigns a partial string
at()	Obtains the character stored at a specified location
Begin()	Returns a reference to the start of a string
capacity()	Gives the total elements that can be stored.
compare()	Compares string against the invoking string
empty()	Returns true if the string is empty; Otherwise returns false
end()	Returns a reference to the end of a string
erase()	Removes characters as specified
find()	Searches for the occurrence of a specified substring
insert()	Inserts characters at a specified location
length()	Gives the number of elements in a string
max_size()	Gives the maximum possible size of a string object in a give system
replace()	Replace specified characters with a given string
resize()	Changes the size of the string as specified
size()	Gives the number of characters in the string
swap()	Swaps the given string with the invoking string

Table 15.3 *Operators for string objects*

Operator	*Meaning*
=	Assignment
+	Concatenation
+=	Concatenation assignment
= =	Equality
!=	Inequality
<	Less than
<=	Less than or equal
>	Greater than
>=	Greater than or equal
[ ]	Subscription
<<	Output
>>	Input

```
string s1;              // Using constructor with no argument
string s2("xyz");       // Using one-argument constructor
s1 = s2;                // Assigning string objects
s3 = "abc" + s2         // Concatenating strings
cin >> s1;              // Reading through keyboard (one word)
getline(cin, s1);       // Reading through keyboard a line of text
```

The overloaded + operator concatenates two string objects. We can also use the operator += to append a string to the end of a string. Examples:

```
s3 += s1;         // s3 = s3 + s1
s3 += "abc";      // s3 = s3 + "abc"
```

The operators << and >> are overloaded to handle input and output of string objects. Examples:

```
cin >> s2;              // Input to string object (one word)
cout << s2;             // Displays the contents of s2
getline(cin, s2);       // Reads embedded blanks
```

C++ program

```
#include <iostream>
#include <string>

using namespace std;

int main()
{
    // Creating string objects
    string s1;                    // Empty string object
    string s2(" New");            // Using string constant
    string s3(" Delhi");

    // Assigning value to string objects
    s1 = s2;                      // Using string object
    cout << "S1 = " << s1 << "\n";

    // Using a string constant
    s1 = "Standard C++";
    cout << "Now S1 = " << s1 << "\n";
```

```
    // Using another object
    string s4(s1);
    cout << "S4 = " << s4 << "\n\n";

    // Reading through keyboard
    cout << "ENTER A STRING \n";
    cin >> s4;                          // Delimited by blank sp
    cout << "Now S4 = " << s4 << "\n\n";

    // Concatenating strings
    s1 = s2 + s3;
    cout << "S1 finally contains: " << s1 << "\n";

    return 0;
}
```

output

```
S1 =  New
Now S1 = Standard C++
S4 = Standard C++
```



```
ENTER A STRING
COMPUTER CENTRE
Now S4 = COMPUTER

S1 finally contains:  New Delhi
```

Use of member function to modify string object

```
#include <iostream
#include <string>

using namespace std;

int main()
{
   string s1("12345");
   string s2("abcde");

   cout << "Original Strings are: \n";
   cout << "S1: " << s1 << "\n";
   cout << "S2: " << s2 << "\n\n";

   // Inserting a string into another
   cout << "Place S2 inside S1 \n";
   s1.insert(4,s2);
   cout << "Modified S1: " << s1 << "\n\n";
```

```
// Inserting a string into another
cout << "Place S2 inside S1 \n";
s1.insert(4,s2);
cout << "Modified S1: " << s1 << "\n\n";

// Removing characters in a string
cout << "Remove 5 Characters from S1 \n";
s1.erase(4,5);
cout << "Now S1: " << s1 << "\n\n";

// Replacing characters in a string
cout << "Replace Middle 3 Characters in S2 with S1 \n";
```

```
s2.replce(1,3,s1);
cout<<“now s2= “<<s2<<endl;
return 0;
}
```

output

- Original strings are
- S1:12345
- S2:abcde
- Place s2 inside s1
- Modified s1:1234abcde5
- Remove 5 characters from s1
- Now s1:12345
- Replace middle 3 characters in s2 with s1
- Mow s2:a12345e

Relational operators

```
int main()
{
    string s1("ABC");
    string s2("XYZ");
    string s3 = s1 + s2;

    if(s1 != s2)
            cout << "s1 is not equal to s2 \n";
    if(s1 > s2)
            cout << "s1 greater than s2 \n";
    else
            cout << "s2 greater than s1 \n";
    if(s3 == s1 + s2)
            cout << "s3 is equal to s1+s2 \n\n";
    int x = s1.compare(s2);
```

```
    int x = s1.compare(s2);
    if(x == 0)
            cout << "s1 == s2 \n";
    else if(x > 0)
            cout << "s1 > s2 \n";
    else                                // x < 0
            cout << "s1 < s2 \n";

    return 0;
}
```

String characteristics

```
void display(string &str)
{
   cout << "Size = " << str.size() << "\n";
   cout << "Length = " << str.length() << "\n";
   cout << "Capacity = " << str.capacity() << "\n";
   cout << "Maximum Size = " << str.max_size() << "\n";
   cout << "Empty: " << (str.empty() ? "yes" : "no");
   cout << "\n\n";
}

int main()
{
   string str1;

   cout << "Initial status: \n";
   display(str1);

   cout << "Enter a string (one word) \n";
   cin >> str1;
   cout  << "Status now: \n";
   display(str1);
```

Accessing and manipulating characters

```
string s("ONE TWO THREE FOUR");

cout << "The string contains: \n";
for(int i=0;i<s.length();i++)
        cout << s.at(i);                // Display one character
cout << "\nString is shown again: \n";
for(int j=0;j<s.length();j++)
        cout << s[j];

int x1 = s.find("TWO");
cout << "\n\nTWO is found at: " << x1 << "\n";

int x2 = s.find_first_of('T');
cout << "\nT is found first at: " << x2 << "\n";
int x3 = s.find_last_of('R');
cout << "\nR is last found at: " << x3 << "\n";

cout << "\nRetrieve and print substring TWO \n";

cout << s.substr(x1,3);
```

COMPARING AND SWAPPING STRINGS

```
#include <iostream>
#include <string>

using namespace std;

int main()
{
   string s1("Road");
   string s2("Read");
   string s3("Red");
   cout << "s1 = " << s1 << "\n";
   cout << "s2 = " << s2 << "\n";
   cout << "s3 = " << s3 << "\n";

   int x = s1.compare(s2);
   if(x == 0)
```

```
        cout << "s1 == s2" << "\n";
else if(x > 0)
        cout << "s1 > s2" << "\n";
else
        cout << "s1 < s2" << "\n";

int a = s1.compare(0,2,s2,0,2);
int b = s2.compare(0,2,s1,0,2);
int c = s2.compare(0,2,s3,0,2);
int d = s2.compare(s2.size()-1,1,s3,s3.size()-1,1);

cout << "a = " << a << "\n" << "b = " << b << "\n";
cout << "c = " << c << "\n" << "d = " << d << "\n";

cout << "\nBefore swap: \n";
cout << "s1 = " << s1 << "\n";
cout << "s2 = " << s2 << "\n";
s1.swap(s2);
cout << "\nAfter swap: \n";
cout << "s1 = " << s1 << "\n";
```